

discretes: Representing enumerable numeric series in R without listing their members

Vincenzo Coia ¹ and Carlo De Michele ¹

¹ C2E Lab, Department of Civil and Environmental Engineering (DICA), Politecnico di Milano, Milan, Italy  Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 15 September 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Many computations need to describe a set of numbers that is countable but too large to store. A clear example is the set of values a discrete random variable can take: a Poisson variable can be any of 0, 1, 2, ..., without end. A computer cannot hold infinitely many numbers, and in R there is no built-in way to write down $0:\text{Inf}$ and work with it. The **discretes** package represents such sets — it calls them *numeric series* — by storing the rule that generates a series rather than its members. A series can be traversed one value at a time, tested for membership, counted over an interval, combined with other series, and transformed by arithmetic, all without ever listing its elements. Where a choice has to be made, discretes follows the rules R already uses for ordinary numeric vectors, so that a series behaves the way a vector would if a vector could be infinite.

Statement of need

discretes was written for the probaverse project (Coia & others, 2025), a set of R packages that treat probability distributions as first-class objects. A central goal there is to support any distribution a user might construct, not only named families. Most named distributions with a discrete component — Poisson, geometric, hypergeometric — live on the natural numbers (Johnson et al., 2005), and it is tempting to assume every discrete support does. That assumption breaks as soon as distributions are shifted, scaled, or combined, which is exactly what applied modelling requires. To reason about an arbitrary distribution we have to answer concrete questions about its support: What is the next admissible value above a cutoff? Are there finitely or infinitely many outcomes between two points? Is a particular number an outcome at all?

Base R has finite numeric vectors and sequence helpers (`seq()`, `:`), but no type for an infinite enumerable set that can be transformed and queried consistently. The usual workarounds — store a long prefix of the values, hard-code the support of each named distribution, or quietly truncate the set — are fragile, and they tend to fail near the values that matter most, such as a limit the series approaches. discretes provides this missing abstraction, and although it was written for probability, it is a general-purpose tool for enumerable numeric series.

State of the field

Several R packages address neighbouring problems. Zseq (You, 2025) supplies named integer sequences such as the Fibonacci and prime numbers, but not a general way to construct and transform series. sets (Meyer & Hornik, 2023) implements finite set operations and abstract set algebra, with no notion of successor and predecessor along the number line. set6 (Sonabend & Kiraly, 2021) offered object-oriented infinite sets but is no longer on CRAN and was not

built around agreement with R's numeric-vector behaviour. `peruse` (Goldsmith, 2021) iterates over general sequences with a well-defined starting value, with less attention to arithmetic manipulation and limit points. `discretes` was written as its own layer rather than as a wrapper over these because its target object is a locally enumerable numeric series — closed under operations commonly applied to numeric vectors, and explicit about the floating-point cases (signed zero, infinities) that those operations expose.

Software design

What the package represents

`discretes` does not try to represent every countable set of numbers. Infinitely dense sets such as the rationals are excluded, because they have no well-defined “next” value: every real number is a limit point of the rationals. What `discretes` does handle is series whose values are isolated except at a finite number of limit points. Following the package's shorthand, we call these limit points *sinks*. The natural numbers $0, 1, 2, \dots$ have a single sink, at infinity. The series $1, 1/2, 1/4, \dots$ has a sink at 0, approached from the right. Allowing finitely many sinks covers the supports that arise in practice while keeping every series finitely describable. These are sets, so each value appears once; but unlike an arbitrary set, their values sit in order along the number line and are spaced apart — isolated except at the finitely many sinks. That order is what lets us ask for the “next” or “previous” value.

Two base series, then manipulation

Every series is built from one of two base forms. The first is an *arithmetic series*, stored as a seed value, a spacing, and the number of steps to take left and right of the seed; either count may be infinite, which is how a base series acquires a sink. The integers, the natural numbers, and any evenly spaced grid are arithmetic series. The second is an explicit *numeric vector*. A finite set of values often has no generating pattern — the outcomes can be scattered anywhere among the reals — so the vector itself is stored and treated as a series.

From these two forms, new series are built by manipulation: union of several series; the arithmetic operations $+$, $-$, $*$, $/$, and exponentiation; functions such as $\log$ and $\exp$; trimming a series to an interval; and arbitrary transformations, including non-monotonic ones, through `dsct_transform()`. This is a deliberately small set of primitives, but composing them reaches a large majority of the numeric series one actually needs.

It does not reach all of them. Sequences with their own bespoke logic, such as the Fibonacci numbers, could have been supported but were left out on purpose: the aim was to cover the common cases well, not to chase every constructible set. (The Fibonacci example also makes a point about sets. Although the sequence is written $1, 1, 2, 3, 5, \dots$, a set holds each value once, so the two discrete values above 0 are 1 and 2, not 1 and 1.)

This mirrors how the rest of `probaverse` is built. The `distplyr` package supplies base distributions and `distplyr` manipulates and combines them; the named families act like a few axes through the space of all distributions, and manipulation fills in the space between them. `discretes` applies the same idea one level down, to the numeric series that describe distribution supports.

Querying without enumerating

A numeric series answers a fixed set of questions, and these define the contract that every series type must satisfy: `next_discrete()` and `prev_discrete()` to traverse, `num_discretes()` to count the values in an interval, `has_discretes()` to test membership, and `has_sink_at()` / `has_sink_in()` to report limit points.

85 The same contract is what makes manipulation work. A transformation is itself a new series
 86 whose definition happens to refer to another series. Because it is a series, it implements
 87 the same contract, and it answers a query by querying the series underneath it. Wrapping
 88 manipulations on top of one another builds a ladder of these objects; a query descends the
 89 ladder until it reaches a base series — an arithmetic series or a vector — where the question
 90 can be answered directly. The package uses S3 method dispatch for this, with one method
 91 per series type for each generic. Traversal is defined directly for each type rather than, say,
 92 treating a backward step as a forward step on a negated series, which would invite infinite
 93 recursion on composed objects.

94 As a concrete example, take a Poisson variable X , whose support is the natural numbers
 95 $0, 1, 2, \dots$, and form the transformed support of $1/2^X$. The transformation acts on the *support*,
 96 and it turns the sink at infinity into a sink at 0:

```
support <- 1 / 2^natural0()      # support of 1 / 2^X, with X ~ Poisson
has_sink_at(support, 0)         # TRUE: the transform creates a sink at 0
num_discretes(support, 0.1, 1)  # 4: outcomes in [0.1, 1], none enumerated
```

97 Congruence with base R

98 One rule runs through the package: a series should behave exactly as a numeric vector would.
 99 Whenever an operation has an established meaning for vectors, discretes defers to it rather
 100 than inventing a parallel convention. R defines 0^0 as 1, for instance, so a series does the same,
 101 and results keep R's integer type where a vector would.

102 Making this precise requires separating two views of a series: its *encoding* — the parameters
 103 that define it, such as a seed, spacing, and step counts for arithmetic series, or the original
 104 vector — and its *expression*, the actual numeric values R produces when the series is realized.
 105 Operations that need a concrete answer are delegated to R on the expressed values.

106 Signed zero

107 The clearest case where encoding and expression must be kept apart is signed zero. In IEEE-754
 108 arithmetic R has both $+0$ and -0 ; `identical(0, -0)` is TRUE, and mathematically there is
 109 only one zero, so a series containing both still reports a single value. But the two are not
 110 interchangeable: $1 / 0$ is Inf while $1 / -0$ is -Inf (Goldberg, 1991). (The difference is
 111 visible in the raw bytes: `writeBin(-0, raw())` sets the sign bit, giving 00 00 00 00 00 00
 112 00 80, as opposed to all zeros for $+0$.) A series that expressed itself as a single 0 would
 113 invert to a single infinity and silently lose the other. discretes therefore tracks the latent signs
 114 with `has_positive_zero()` and `has_negative_zero()`: the set $\{0, -0\}$ is one value when
 115 enumerated, but its reciprocal is the two-value series $\{-\infty, +\infty\}$ (infinities are themselves
 116 allowed as discrete values). Getting this right is not pedantry — it is what lets inversion of a
 117 support agree with the inversion R would perform on the corresponding vector.

118 Research impact statement

119 discretes is released on CRAN under the MIT license, documented with a pkgdown site and
 120 four vignettes — two introducing the construction and querying of series, two covering the
 121 finer points of numerical tolerance and signed zero — and tested with an extensive testthat
 122 suite that checks the series' behavioural contracts, including the sink and signed-zero cases
 123 (Coia & De Michele, 2026). It is a dependency of distionary (Coia, 2026), the core package of
 124 probaverse, which since version 0.2.0 uses discretes to represent distribution supports so that
 125 downstream code can combine models and run computations on precise outcomes rather than
 126 numerically derived values. Infinite-but-regular supports are routine in discrete modeling, and
 127 handling their limits and floating-point edge cases correctly is a prerequisite for dependable
 128 simulation and inference.

AI usage disclosure

Generative AI assisted with drafting and editing this manuscript. The authors reviewed every technical statement and design description for accuracy. Generative AI was not used to make the core design decisions for the *discretes* software itself, nor to decide on the content of this article.

Acknowledgements

Development of *discretes* was supported by the European Space Agency. The need for the package arose from work on the *probaverse* project. The authors thank colleagues who reviewed package behaviour and documentation during open development.

References

- Coia, V. (2026). *distionary: Create and evaluate probability distributions*. <https://CRAN.R-project.org/package=distionary>
- Coia, V., & De Michele, C. (2026). *discretes: Discrete numeric series*. <https://CRAN.R-project.org/package=discretes>
- Coia, V., & others. (2025). *Probaverse: Tools for probabilistic modeling and inference in R*. Software project. <https://probaverse.com/>
- Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1), 5–48. <https://doi.org/10.1145/103162.103163>
- Goldsmith, J. (2021). *peruse: A tidy API for sequence iteration and set comprehension*. <https://CRAN.R-project.org/package=peruse>
- Johnson, N. L., Kemp, A. W., & Kotz, S. (2005). *Univariate discrete distributions* (3rd ed.). Wiley.
- Meyer, D., & Hornik, K. (2023). *sets: Sets, generalized sets, customizable sets and intervals*. <https://CRAN.R-project.org/package=sets>
- Sonabend, R., & Kiraly, F. (2021). *set6: R6 mathematical sets interface*. <https://CRAN.R-project.org/package=set6>
- You, K. (2025). *Zseq: Integer sequence generator*. <https://CRAN.R-project.org/package=Zseq>